

```

import java.util.*;
Public class MergeSort
{
    Public static void print(int arr[])
    {
        System.out for(int i=0; i<arr.length; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }

    Public static void mergesort(int arr[], int si, int ei)
    {
        if (si >= ei)
        {
            return;
        }
        int mid = si + (ei - si) / 2;
        mergesort(arr, si, mid);
        mergesort(arr, mid + 1, ei);
        merge(arr, mid, si, ei);
    }

    Public static void merge(int packagevalue[], int si,
                             int mid, int ei)
    {
        int temp[] = new int(ei - si + 1);

        int i = si;
        int j = mid + 1;
        int k = 0;

```

```

        while (i <= mid && j <= ei)
        {
            if (packagevalue[i] < packagevalue[j])
            {
                temp[k] = packagevalue[i];
                k++;
                i++;
            }
            else
            {
                temp[k] = packagevalue[j];
                k++;
                j++;
            }
        }
        while (i <= mid)
        {
            temp[k] = packagevalue[i];
            k++;
            i++;
        }
        while (j <= ei)
        {
            temp[k] = packagevalue[j];
            k++;
            j++;
        }
        for (k = 0; i = si; k < temp.length; k++, i++)
        {
            packagevalue[i] = temp[k];
        }
    }
}

```

```

public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter the number of placed student = ");
    int N = sc.nextInt();
    System.out.print("Enter the package values = ");
    int packageValues[] = new int[N];
    for (int i = 0; i < N; i++)
    {
        packageValues[i] = sc.nextInt();
    }
    mergesort(packageValues, 0, N-1);
    print(packageValues);
}

```

```

Search Project Run Window Help
SumOfTwoNum... ArrayPassing... LinearSearch... GetLargestN... BinaryS...
6 public static void print(int arr[])
7 {
8     System.out.print("Enter the package in ascending order=");
9     for(int i=0; i<arr.length; i++)
10     {
11         System.out.print(arr[i] + " ");
12     }
13 }
14
15 public static void mergesort(int packageValues[], int si, int ei)
16 {
17     if(si>ei)
18     {
19         return;
20     }
21     int mid = (si+ei)/2;
22     mergesort(packageValues, si, mid);
23     mergesort(packageValues, mid+1, ei);
24     merge(packageValues, si, mid, ei);
25 }
26
27 public static void merge(int packageValues[], int si, int mid, int ei)
28 {
29     int temp[] = new int[ei-si+1];
30
31     int i=si;
32     int j=mid+1;
33     int k=0;
34
35     while(i<=mid && j<=ei)
36     {
37         if(packageValues[i]<packageValues[j])
38         {
39             temp[k]=packageValues[i];

```

Console X

```

<terminated> Mergeshort [Java Application] C:\Users\Suraj Yadav\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot
Enter the Number of placed Student=3
Enter the package value=450
750
890
Enter the package in ascending order=450, 750, 890,

```

import java.util.*;

public class ProcessingTime

public static void merge(int times[], int si, int mid,
int ei)

int temp[] = new int[ei-si+1];

int i = si;

int j = mid+1;

int k = 0;

while(i <= mid & j <= ei)

if (times[i] <= times[j])

temp[k] = times[i];

i++;

k++;

else

temp[k] = times[j];

k++;

j++;

}

while(i <= mid)

temp[k] = times[i];

k++;

i++;

while(j <= ei)

temp[k++] = times[j++];

for(k = 0; k < temp.length; k++)

times[si+k] = temp[k];

public static void mergeSort(int times[], int si, int ei)

if (si >= ei)

return

int mid = si + (ei - si) / 2;

mergeSort(times, si, mid);

mergeSort(times, mid+1, ei);

merge(times, si, mid, ei);

public static void main(String[] args)

Scanner sc = new Scanner(System.in);

System.out.print("Enter the Integer N=");

int N = sc.nextInt();

int[] times = new int[N];

System.out.print("Enter the N space-separated
processing times = ");


```

{
    times[i] = sc.nextInt();
}
mergesort(times, 0, N-1);

System.out.println("Sorted processing Times");

for (int val : times)
{
    System.out.print(val + " ");
}
System.out.println();

double median;
if (N % 2 == 1)
{
    median = times[N/2];
}
else
{
    median = (times[N/2 - 1] + times[N/2]) / 2.0;
}

int count = 0;
for (int val : times)
{
    if (val > median)
    {
        count++;
    }
}

System.out.println("Order greater than median: " + count);

int diff = times[N-1] - times[0];
System.out.println("Difference (max - min) " + diff);
}
}

```

```

1 package Divideandconquer;
2 import java.util.*;
3
4 public class ProcessingTime {
5
6
7     public static void merge(int[] times, int si, int mid, int ei)
8     {
9         int[] temp = new int[ei - si + 1];
10        int i = si, j = mid + 1, k = 0;
11
12        while (i <= mid && j <= ei) {
13            if (times[i] <= times[j]) {
14                temp[k++] = times[i++];
15            } else {
16                temp[k++] = times[j++];
17            }
18        }
19        while (i <= mid) temp[k++] = times[i++];
20        while (j <= ei) temp[k++] = times[j++];
21
22        for (k = 0; k < temp.length; k++) {
23            times[si + k] = temp[k];
24        }
25    }
26
27    // Merge Sort
28    public static void mergesort(int[] times, int si, int ei)
29    {
30        if (si < ei) {
31            int mid = (si + ei) / 2;
32            mergesort(times, si, mid);
33            mergesort(times, mid + 1, ei);
34            merge(times, si, mid, ei);
35        }
36    }
37
38    // Main Method
39    public static void main(String[] args) {
40        Scanner sc = new Scanner(System.in);
41        System.out.print("Enter the Integer N= ");
42        int N = sc.nextInt();
43        System.out.print("Enter the N space-separated processing times.= ");
44        int[] times = new int[N];
45        for (int i = 0; i < N; i++) {
46            times[i] = sc.nextInt();
47        }
48        mergesort(times, 0, N-1);
49        System.out.println("Sorted Processing Times:");
50        for (int i = 0; i < N; i++) {
51            System.out.print(times[i] + " ");
52        }
53        System.out.println();
54        double median = findMedian(times, N);
55        System.out.println("Median Processing Time: " + median);
56        int count = countGreater(times, median, N);
57        System.out.println("Orders greater than median: " + count);
58        int diff = times[N-1] - times[0];
59        System.out.println("Difference (Max - Min): " + diff);
60    }
61
62    // Find Median
63    public static double findMedian(int[] times, int N) {
64        mergesort(times, 0, N-1);
65        double median;
66        if (N % 2 == 1) {
67            median = times[N/2];
68        } else {
69            median = (times[N/2 - 1] + times[N/2]) / 2.0;
70        }
71        return median;
72    }
73
74    // Count Greater than Median
75    public static int countGreater(int[] times, double median, int N) {
76        int count = 0;
77        for (int i = 0; i < N; i++) {
78            if (times[i] > median) {
79                count++;
80            }
81        }
82        return count;
83    }
84
85    // Difference (Max - Min)
86    public static int findDiff(int[] times, int N) {
87        return times[N-1] - times[0];
88    }
89
90 }

```

Console X

<terminated> ProcessingTime [Java Application] C:\Users\Suraj\p2\poo\plugins\org.eclipse.justj.openjdk.hotspot

Enter the Integer N=5

Enter the N space-separated processing times.=23 32 45 56 78

Sorted Processing Times:
23 32 45 56 78

Median Processing Time: 45.0

Orders greater than median: 2

Difference (Max - Min): 55

QuickSort 2 Question-1

```
import java.util.*;

public class ServerResponseQuickSort {

    public static void printArray(int arr[])
    {
        System.out.print("Print the Response time in  
    ascending order=");
        for (int num : arr)
        {
            System.out.print(num + " ");
        }
    }

    public static void quicksort(int arr[], int low, int  
        high)
    {
        if (low >= high)
        {
            return;
        }
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi - 1);
        quicksort(arr, pi + 1, high);
    }

    public static void partition(int arr[], int low,  
        int high)
    {
        int pivot = high;
        int i = low - 1;
```

```
for (int j = low; j < high; j++)
```

```
    if (arr[j] <= pivot)
    {
        i++;
        int temp = arr[j];
        arr[j] = arr[i];
        arr[i] = temp;
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
```

```
public static void main(String [] args)
{
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
    {
        arr[i] = sc.nextInt();
    }
    quicksort(arr, 0, n - 1);
    printArray(arr);
}
```

```
Explorer X
SumOfTwoNumb... ArrayPassing... LinearSearch... GetLargestN... BinarySearch...
4 public class ServerResponseQuickSort {
5
6
7
8 public static void printArray(int[] arr) {
9
10     System.out.print("Print the Response time in ascending order =");
11     for (int num : arr) {
12         System.out.print(num + " ");
13     }
14 }
15
16
17 public static void quickSort(int[] arr, int low, int high) {
18     if (low < high) {
19
20         int pi = partition(arr, low, high);
21
22         quickSort(arr, low, pi - 1);
23         quickSort(arr, pi + 1, high);
24     }
25 }
26
27
```

Console X

<terminated> ServerResponseQuickSort [Java Application] C:\Users\Suraj Yadav\p2\poc\plugins\org.eclipse.jst.openjdk.hot

Enter the N=6

Enter the element in array=

120

45

78

200

60

95

Print the Response time in ascending order =45 60 78 95 120 200

QuickSort Question 2.6)

```
import java.util.*;
public class TradeValuesQuickSort
{
    public static void quick(int[] arr, int low,
                             int high)
    {
        if (low < high)
        {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    public static int partition(int[] arr, int low,
                                int high)
    {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++)
        {
            if (arr[j] >= pivot)
            {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }
}
```

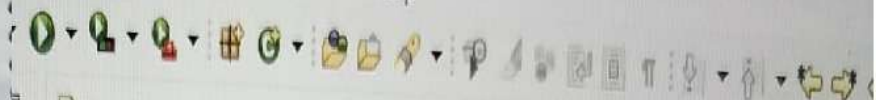
```
int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;
return i + 1;
}

public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    int trades[] = new int[N];
    for (int i = 0; i < N; i++)
    {
        trades[i] = sc.nextInt();
    }
    quickSort(trades, 0, N - 1);

    System.out.println("Sorted Trade Values (Descending)");
    for (int val : trades)
    {
        System.out.print(val + " ");
    }
    System.out.println();

    System.out.println("Top 5 Trade Values");
    int limit = Math.min(5, N);
    int sumTop5 = 0;
```

Search Project Run Window Help



```
SumOfTwoNumb... ArrayPassing... LinearSearc... GetLargestN...
1 package Divideandconcer;
2
3 import java.util.*;
4
5 public class TradeValuesQuickSort {
6
7     // Quick Sort function
8     public static void quickSort(int[] arr, int low, int high) {
9         if (low < high) {
10             int pi = partition(arr, low, high);
11             quickSort(arr, low, pi - 1);
12             quickSort(arr, pi + 1, high);
13         }
14     }
15
16     // Partition method (descending order)
17     public static int partition(int[] arr, int low, int high) {
18         int pivot = arr[high];
19         int i = low - 1;
20
21         for (int j = low; j < high; j++) {
```

Console X

<terminated> TradeValuesQuickSort (Java Application) C:\Users\Suraj Yadav\p2\pool\plugins\org.ecl

Enter the Integer N=8

Enter the Element in array=500

1200

700

2000

900

1500

3000

2500

Sorted Trade Values (Descending):

3000 2500 2000 1500 1200 900 700 500

Top 5 Trade Values:

3000 2500 2000 1500 1200

Average of Top 5: 2040.0

Overall Average: 1537.5

Count > Overall Average: 3